

Kỹ thuật Lập trình Python cho Phân tích Dữ liệu



1



Nội dung

1. Các kiểu dữ liệu cơ sở, toán tử trong Python
2. Cấu trúc rẽ nhánh
3. Cấu trúc lặp
4. Hàm – hàm lambda
5. Pandas
6. Numpy
7. Các thư viện khác ... (Sẽ học trong từng bài phân tích có liên quan)

...

2

2



Các kiểu dữ liệu cơ sở trong Python

1. Kiểu số (Number)
2. Kiểu chuỗi (String)
3. Kiểu danh sách (List)
4. Kiểu tuple /'tu:.pəl/, /'tʌ.pəl/
5. Kiểu từ điển (Dictionary)
6. Kiểu tập hợp (Set)
7. Kiểu Boolean
8. Giá trị None

...

3

3



Kiểu số: int – float – int() – float()

Giới thiệu

• int

- 98
- 15
- 5_600_000
- ...

• float

- 5.9
- 3.9
- 5_600_500.678
- ...

• complex: dạng $a + bj$

...

4

4



Kiểu số: int – float – int() – float()

Các hàm có sẵn

- int() - float()
- max() - min()
- sum(iterable)
- sorted(iterable)
- round(x, n)
- pow(x, y)

import math

- math.sqrt(x)

...

5

5



Kiểu chuỗi – str – str()

Giới thiệu

```
s1 = 'P. Sài Gòn, Tp. HCM'
```

```
s2 = "P. Sài Gòn, Tp. HCM"
```

index

```
#index +      012345...          17
          s =  'P. Sài Gòn, Tp. HCM'
#index -     -18      ...          -1
```

...

6

6



Kiểu chuỗi – str – str()

Các hàm có sẵn

- str()
- len(s)
- format() hoặc f-string

Phương thức liên quan

- s.split(sep) / s.rsplit(sep)
- s.replace(old, new)

...

7

7



Kiểu danh sách – list – list()

Giới thiệu

```
sv = [369, 'Nguyễn Văn A', 7.8, 92]
```

```
sv = [  
    369,  
    'Nguyễn Văn A',  
    7.8,  
    92  
]
```

#index +	0	1	2	3
	sv = [369,	'Nguyễn Văn A',	7.8,	92]
#index -	-4	-3	-2	-1

...

8

8



Kiểu danh sách – list – list()

index

```
#index +      0          1          2      3
        sv = [369, 'Nguyễn Văn A', 7.8, 92]
#index -      -4          -3          -2     -1
```

```
sv[2]
```

```
sv[-3]
```

```
sv[1] = "Nguyễn Văn B"
```

9

9



Kiểu danh sách – list – list()

Các hàm có sẵn

- list(iterable)
- len(lst)
- sum(lst)
- max(lst), min(lst)
- sorted(lst, reverse=True)
- any(lst) trả về True nếu có ít nhất một phần tử True.
- all(lst) trả về True nếu tất cả phần tử đều True.

```
diem = [7.3, 9.5, 0, 4.6, 9.1]
```

```
any(diem) # True
```

```
all(diem) # False
```

10

10



Kiểu danh sách – list – list()

Phương thức liên quan

- list.append(x)
- list.extend(iterable)
- list.insert(i, x)
- list.pop()
- list.remove(x)
- list.clear()
- list.sort(reverse=True)

...

11

11



Kiểu tuple – tuple – tuple()

Giới thiệu

```
sv = (369, 'Nguyễn Văn A', 7.8, 92)

sv = (
    369,
    'Nguyễn Văn A',
    7.8,
    92
)
```

#index +	0	1	2	3
sv =	(369,	'Nguyễn Văn A',	7.8,	92)
#index -	-4	-3	-2	-1

...

12

12



Kiểu tuple – tuple – tuple()

Giới thiệu

```
#index +      0              1              2      3
        sv = 369, 'Nguyễn Văn A', 7.8, 92
#index -      -4              -3              -2     -1
```

- Không thay đổi được sau khi tạo.

```
sv[2]
sv[-3]

sv[1] = "Nguyễn Văn B"
```

13

13



Kiểu dict – dict – dict()

Giới thiệu

```
sv = {
    'ma_sv': 369,
    'ho_ten': 'Nguyễn Văn A',
    'diem_tb': 7.8,
    'diem_rl': 92
}
```

```
sv = {'ma_sv': 369, 'ho_ten': 'Nguyễn Văn A', 'diem_tb': 7.8, 'diem_rl': 92}
```

14

14



Kiểu dict – dict – dict()

index - key

```
sv['ho_ten']  
  
sv['diem_tb'] = 8.1
```

...

15

15



Kiểu dict – dict – dict()

Các hàm có sẵn

- dict(): Tạo dict từ iterable hoặc mapping.
- len(dict)
- sorted(dict)
- any(dict)
- all(dict)

...

16

16



Kiểu dict – dict – dict()

Phương thức liên quan

- `d.keys()` Lấy danh sách key.
- `d.values()` Lấy danh sách value.
- `d.items()` Lấy danh sách cặp (key, value).
- `d.get(key, default)` Lấy value theo key, nếu không có thì trả về default.
- `d.setdefault(key, default)` Giống get, nhưng nếu key ko tồn tại thì thêm vào.

...

17

17



Kiểu dict – dict – dict()

Ứng dụng trong phân tích dữ liệu

- Biểu diễn dữ liệu có cấu trúc: dạng bảng, JSON
- Áp dụng nhiều trong `pandas.DataFrame`
- Hỗ trợ gom nhóm, đếm tần suất, ánh xạ, tra cứu nhanh.
- Trong phân tích dữ liệu:
 - dict thường là bước trung gian
 - Sau đó, chuyển thành `DataFrame` (pandas) hoặc array (NumPy) để xử lý sâu hơn.

...

18

18



Kiểu dict – dict – dict()

```
value_count = {'fwd': 98, 'rwd': 23, '4wd': 63}
value = ['rwd', '4wd', 'rwd', 'fwd']
value_new = [value_count[v] for v in value]

value_new
```

...

19

19



Kiểu tập hợp – set – set()

Giới thiệu

- Là tập hợp các phần tử duy nhất, không trùng lặp, không có thứ tự.

```
s = {34, 89, 'Python', 56, 9.5, 'Java'}
```

...

20

20



Kiểu tập hợp – set – set()

Các hàm có sẵn

- set(iterable)
- len(s)
- sorted(s)
- max(s), min(s)
- sum(s)
- any(s), all(s)

...

21

21



Kiểu tập hợp – set – set()

Phương thức liên quan

- | | |
|----------------------|-------------------------------------|
| • s.add(x) | |
| • s.clear() | |
| • s.update(iterable) | Thêm nhiều phần tử |
| • s.remove(x) | Xóa x (lỗi nếu không tồn tại) |
| • s.discard(x) | Xóa x (không lỗi nếu không tồn tại) |
| • s.pop() | Xóa một phần tử ngẫu nhiên |

...

22

22



Kiểu tập hợp – set – set()

Ứng dụng trong phân tích dữ liệu

- Loại bỏ trùng lặp

```
data = [30, 40, 40, 10, 20, 20, 50]
data_new = set(data)
```

- Giá trị chung giữa hai cột dữ liệu

```
a_column = {'Honda', 'BMW', 'Mec'}
b_column = {'Suzuki', 'Honda',
            'Toyota'}

a_column & b_column
```

23

23



Kiểu bool – bool()

Giới thiệu

- Chỉ có 2 giá trị: True, False

#	số	str, list, tuple, dict, set	
True	# khác 0	Không rỗng	
False	# 0	Rỗng	None

24

24



Giá trị None

Giới thiệu

- Giá trị None đại diện giá trị không có giá trị hoặc giá trị không xác định.

```
name = None  
print(name)
```

```
name = 'Văn B'  
print(name)
```

```
value = 5
```

```
if value is None:  
    print("Value có giá trị None")  
else:  
    print("Value = ", value)
```

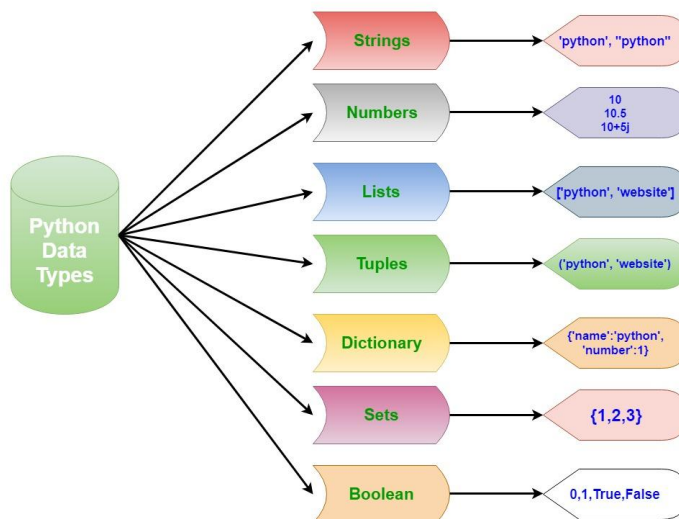
- Trong phân tích dữ liệu, None là một trong những giá trị đại diện cho dữ liệu bị khuyết (missing data).

25

25



Tóm tắt



26

26



Toán tử

1. Toán tử số học
2. Toán tử gán
3. Toán tử so sánh
4. Toán tử logic
5. Toán tử kiểm tra tồn tại in và not in
6. Độ ưu tiên

...

27

27



TOÁN TỬ

Toán tử số học

- +** ...
- ...
- *** ...
- /** ...
- %** Chia lấy dư
- //** Lấy nguyên
- **** Lũy thừa

...

28

28



TOÁN TỬ

Toán tử gán

- = Phép gán
- += Cộng, rồi gán lại $x = 5$ $x += 1$
- = Trừ, rồi gán lại
- *= Nhân, rồi gán lại
- /= Chia, rồi gán lại
- %= Chia lấy dư, rồi
- //= Chia lấy nguyên, rồi gán lại gán lại
- **= Tính lũy thừa, rồi gán lại

...

29

29



TOÁN TỬ

Toán tử so sánh

- > Lớn hơn
- < Nhỏ hơn
- >= Lớn hơn hoặc bằng
- <= Nhỏ hơn hoặc bằng
- == Bằng
- != Khác (Không bằng nhau)

⇒ Kết quả sau khi thực hiện trả về kiểu **bool**

...

30

30



TOÁN TỬ

Toán tử logic

and phép và

or phép hoặc

not phép phủ định

⇒ Kết quả sau khi thực hiện trả về kiểu **bool**

...

31

31



TOÁN TỬ

*Toán tử thành viên (membership operators) **in** và **not in***

- Sử dụng để kiểm tra xem một giá trị có thuộc (hoặc không thuộc) một chuỗi, danh sách, tuple, dict (=>key), tập hợp hay không.

...

32

32



TOÁN TỬ

Độ ưu tiên cơ bản

- Dùng dấu (...) thể hiện độ ưu tiên nếu muốn.
- Độ ưu tiên **cơ bản** được thể hiện theo thứ tự sau:

SỐ HỌC > SO SÁNH > LOGIC > GÁN

*Học viên tìm hiểu thêm các độ ưu tiên bên trong của **SỐ HỌC, SO SÁNH, LOGIC***

...

33

33



Cấu trúc rẽ nhánh

- 1. Dạng #1:** **if**
- 2. Dạng #2:** **if – else**
- 3. Dạng #3:** **if - elif - else**

...

34

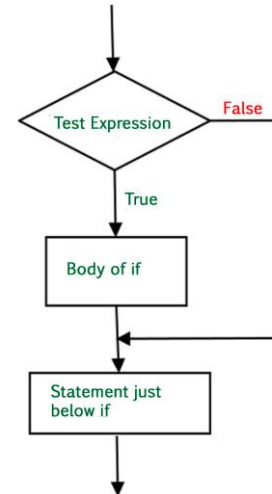
34



if

Cú pháp if

```
if <biểu thức điều kiện> :  
    <Lệnh hoặc khối lệnh>
```



...

35

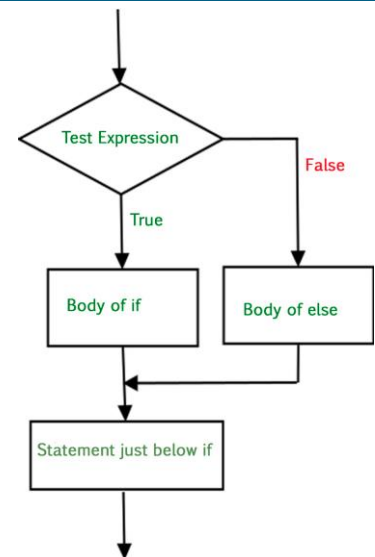
35



if - else

Cú pháp if - else

```
if <biểu thức điều kiện> :  
    <Lệnh hoặc khối lệnh>  
else :  
    <Lệnh hoặc khối lệnh trong else>
```



...

36

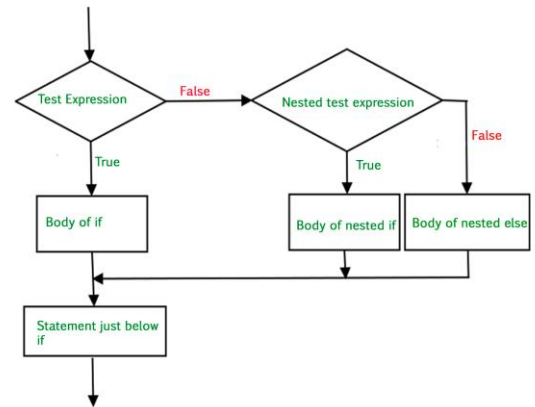
36



if – elif – else

Cú pháp if – elif – else

```
if <điều kiện #1> :  
    <Lệnh hoặc khối lệnh>  
  
elif <điều kiện #2> :  
    <Lệnh hoặc khối lệnh>  
  
elif <điều kiện #3> :  
    <Lệnh hoặc khối lệnh>  
    .....  
    .....  
else :  
    <Lệnh hoặc khối lệnh>
```



37

37



BÀI TẬP

- Viết chương trình mô phỏng trò chơi **kéo - búa - bao** giữa người và máy.

Code it!

Qui ước:

- Kéo > Bao
- Búa > Kéo
- Bao > Búa

Người ra: kéo
Máy ra: bao
=> Kết quả: Người thắng máy

38

38



Cấu trúc lặp

1. Cấu trúc lặp **for**
2. Cấu trúc lặp **while**
3. Lệnh **continue**
4. Lệnh **break**

39

39



for

Cú pháp

```
for <biến lặp> in <iterable> :  
    <Lệnh hoặc khối lệnh>
```

- **<iterable>** đối tượng có thể lặp. Gồm str, list, tuple, dict, set, **range**...

40

40



for

Áp dụng. Cách tạo vòng lặp từ khi biết trước số lần lặp (n lần):

```
for i in range(n) :  
    <Lệnh hoặc khối lệnh>  
    print(i)
```

```
for i in range(start, stop, step) :  
    <Lệnh hoặc khối lệnh>  
    print(i)
```

...

41

41



List Comprehension

- Tạo một danh sách bằng cách kết hợp lệnh for và biểu thức

Cú pháp – còn gọi là cú pháp comprehension

```
new_list = [expression for item in iterable if condition]
```

- expression: biểu thức để tính toán giá trị của từng phần tử trong list mới.
- item: biến được sử dụng để lặp qua từng phần tử trong list ban đầu.
- condition: điều kiện kiểm tra mà các item phải thỏa để được thêm vào new_list

Ví dụ

```
ds = [2, 4, 6, 8, 10]  
  
new_ds = [item - 1 for item in ds]  
  
print(new_ds)
```

...

42

42



Dictionary Comprehension

Cú pháp

{key_expr: value_expr for item in iterable if condition}

```
d = {'fwd': 98, 'rwd': 23, '4wd': 63}
{k: v for k, v in d.items() if v >= 60}
```

...

43

43



Set Comprehension

Cú pháp

{expression for item in iterable if condition}

```
s = {89, 35, 78, 12, 56, 69}
{item for item in s if item >= 50}
```

...

44

44



while

Cú pháp lệnh while

```
while <điều kiện> :  
    <Lệnh hoặc khối lệnh>
```

- Cách nhớ:

“kiểm tra điều kiện trước rồi mới thực hiện lệnh”

...

45

45



while

Ví dụ minh họa

Nhập vào một số dương, nếu không đúng bắt người dùng nhập lại đến khi nào đúng thì thôi (tức là dừng nhập).

```
x = float(input("Nhập x = "))  
  
while x < 0:  
    x = float(input("Nhập lại x = "))  
  
print("Giá trị đã nhập:", x)
```

...

46

46



break

Dùng **break** để thoát **for**

```
for <biến vòng lặp> in <dãy giá trị> :  
    <Lệnh hoặc khối lệnh bên trong vòng lặp>  
    if <điều kiện để dừng vòng lặp> :  
        break  
    <Lệnh hoặc khối lệnh bên trong vòng lặp>
```

...

47

47



continue

Sử dụng **continue** trong **for**

```
for <biến vòng lặp> in <dãy giá trị> :  
    <Lệnh hoặc khối lệnh bên trong vòng lặp>  
    if <điều kiện> :  
        continue  
    <Lệnh hoặc khối lệnh bên trong vòng lặp>
```

...

48

48



Hàm

1. Hàm được tích hợp sẵn
2. Xây dựng hàm
3. Các tham số
4. Hàm lambda

...

49

49



Hàm được tích hợp sẵn

abs()	delattr()	hash()	memoryview()	set()
all()	dict()	help()	min()	setattr()
any()	dir()	hex()	next()	slice()
ascii()	divmod()	id()	object()	sorted()
bin()	enumerate()	input()	oct()	staticmethod()
bool()	eval()	int()	open()	str()
breakpoint()	exec()	isinstance()	ord()	sum()
bytearray()	filter()	issubclass()	pow()	super()
bytes()	float()	iter()	print()	tuple()
callable()	format()	len()	property()	type()
chr()	frozenset()	list()	range()	vars()
classmethod()	getattr()	locals()	repr()	zip()
compile()	globals()	map()	reversed()	__import__()
complex()	hasattr()	max()	round()	

Ref: <https://docs.python.org/3/library/functions.html>

...

50

50



func

Cú pháp

```
def func_name(<para#1, para#2, ..., para#n>):  
    """ docstring: Mô tả hàm """  
  
    <Lệnh hoặc Khối lệnh trong hàm>  
    return x, y, x, ...
```

Chú ý:

Có thể khai báo kèm thêm kiểu dữ liệu cho tham số và kiểu dữ liệu của hàm

...

51

51



func

Ví dụ minh họa

Hàm tính cộng, trừ, nhân, chia của hai số.

```
def tinh(a, b):  
    cong = a + b  
    tru = a - b  
    nhan = a * b  
    chia = a / b  
  
    return cong, tru, nhan, chia  
  
print(tinh(9, 10))
```

```
def tinh(a:int, b:float) -> tuple:  
    cong = a + b  
    tru = a - b  
    nhan = a * b  
    chia = a / b  
  
    return cong, tru, nhan, chia  
  
print(tinh(9, 10))
```

...

52

52



func

Các loại tham số (quan trọng)

```
def func(x, y = 1, *args, **kwargs):  
    ...  
    ...  
    return x, y, args, kwargs
```

...

53

53



lambda

Cú pháp

`lambda <danh_sách_tham_số>: biểu_thức`

```
l = [-2, -7, 3, -9, 6]  
  
list(filter(lambda x: x > 0, l))
```

...

54

54



lambda

Ứng dụng trong phân tích dữ liệu

- Xử lý dữ liệu theo dòng/cột (dùng bên trong apply, map)
- Lọc dữ liệu (dùng bên trong hàm filter)
- Gom nhóm và tính toán (dùng bên trong hàm groupby)
- Sắp xếp tùy chỉnh (dùng bên trong hàm sort_values)
- Chuẩn hóa
- Định dạng

...

55

55



pandas

Giới thiệu

- pandas: thư viện mã nguồn mở, phổ biến, chuyên dùng xử lý và phân tích dữ liệu.
- Series: dạng 1 chiều, có index.
- DataFrame: bảng 2 chiều, gồm dòng và cột, có index cho dòng và cột.

...

56

56



pandas: datatype

- **Kiểu số (Numeric types):** int64, int32, int16, float64, float32... complex
- **Kiểu chuỗi (String/Text):** string (ver 1.0), object
- **Kiểu bool:** True hoặc False
- **Kiểu thời gian – ngày tháng (Date & Time types)**
 - datetime64[ns]: ngày giờ (giây, mili, nano giây), timedelta[ns]: khoảng thời gian
 - Period: khoảng thời gian định kỳ (tháng, quý, năm...).
- **Categorical:** dữ liệu phân loại
- **Kiểu hỗn hợp**
 - object: chứa dữ liệu hỗn hợp (chuỗi, số...),
 - pandas ko xác định đc kiểu thì mặc định là object.

57

57



Series

Giới thiệu

- Series: lưu trữ nhiều giá trị theo 1 chiều
- Các giá trị kèm index (mặc định hoặc tự định nghĩa).

```
s = pd.Series(data=[1, 2, 3, 4, 5, 6, 7, 8])

s = pd.Series(data=[1, 2, 3, 4, 5, 6, 7, 8],
              index=['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
              )
```

58

58



Series: Toán tử

- Toán tử số học
- Toán tử so sánh
- Toán tử logic: $\&$, $|$, $\sim$ $\Rightarrow$ and, or, not
- Toán tử thành viên: in, not in
- Toán tử gán: $+=$, $-=$, $*=$, $/=$, $//=$, $**=$, $\%=$

...

59

59



Series: Toán tử

Minh họa

$s + 2$

$s \geq 3$

$s[(s \leq 3) \mid \sim(s \geq 6)]$

$0 \text{ in } s$

$0 \text{ in } s.\text{values}$

$s1 + s2$

...

60

60



Series: Method

- Chứa nhiều phương thức xử lý và phân tích dữ liệu
- Url: <https://pandas.pydata.org/docs/reference/api/pandas.Series.html>
- GV minh họa trực tiếp

61

61



Dataframe

Giới thiệu

- DataFrame là một bảng 2 chiều gồm dòng (rows) và cột (columns).
- Mỗi cột trong DataFrame là một Series.
- Có index đại diện dòng.
- Có tên column đại diện cho cột.

- GV minh họa trực tiếp

	Name	Team	Number	Position	Age
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

62

62



63



Hỏi - Đáp



64

64